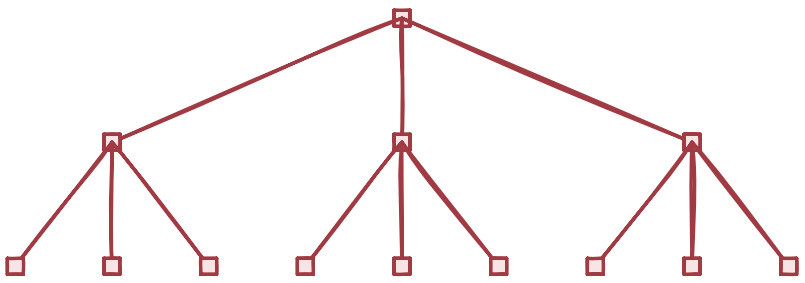
Notes on Cal.com's scheduling algorithms

A booking page has to answer now. Cal.com keeps free-time math exact, then makes each host assignment with a greedy choice.

Two ways to handle future bookings

Try every assignment



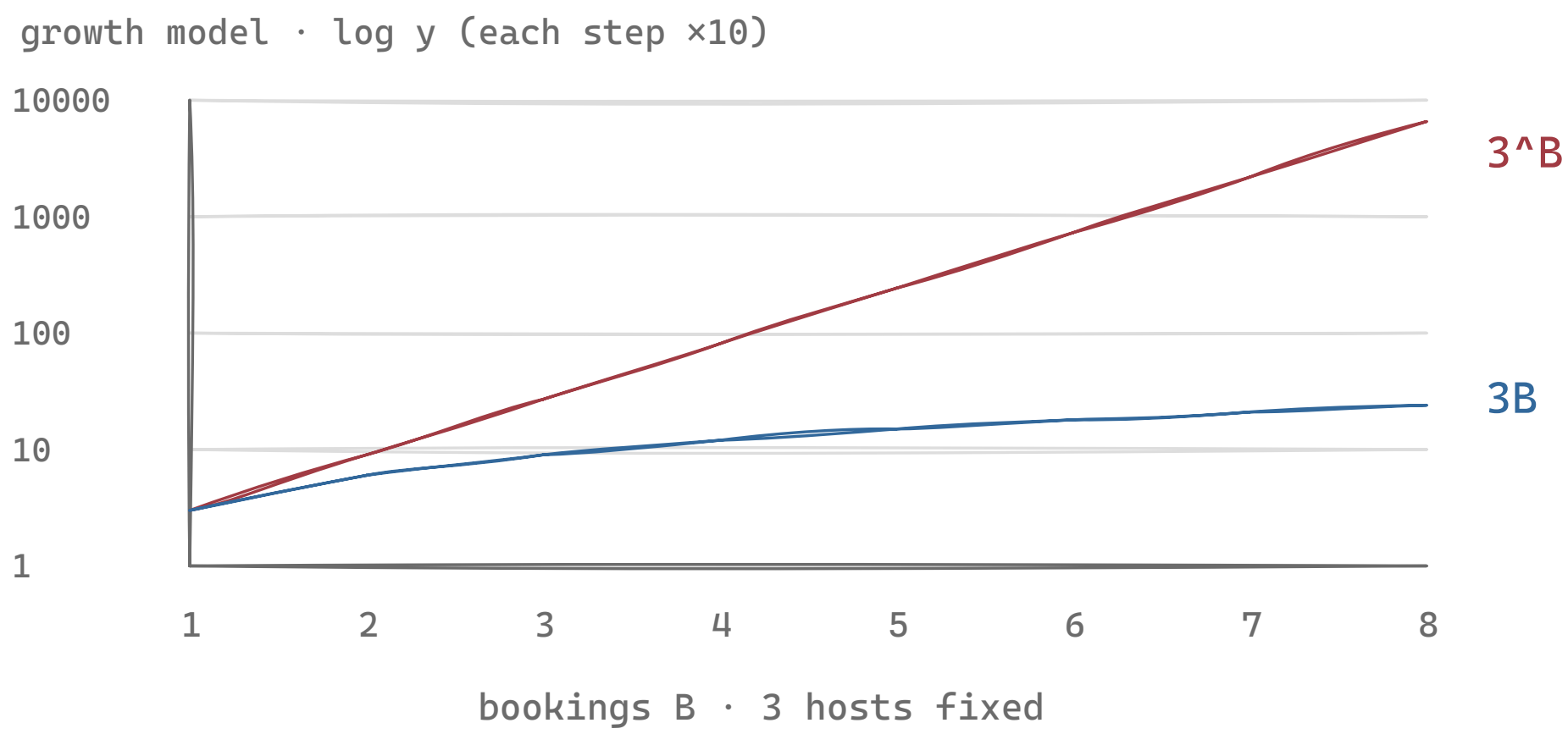
H hosts, B bookings → H^B possibilities

Pick one, move on



O(H) per choice
O(BH) across B bookings

The gap is in how the work grows.



This compares growth, not speed. The blue curve models scan work; constant factors are left out. The red curve counts full assignments.

What I'd remember: exhaustive search can pick the best schedule only for the future requests and objective I've actually modeled. Greedy skips that lookahead. That's why it can answer quickly.

Make the next choice. Then keep it fair.

With weighted distribution on, Cal.com first keeps the available hosts furthest below their target share. Priority and recency come next.

A small example makes the policy clear.

`12 bookings so far · equal weights · target = 4 each`

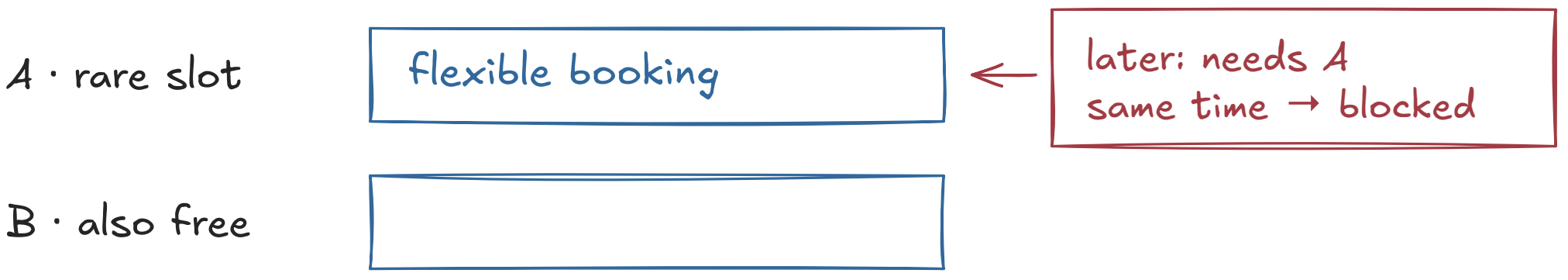


`shortfall = target - actual`

The order is a product choice. Swap the filters, and the same booking can go to someone else. This isn't a global optimum.

The future it doesn't see

Separate example: two requests want 10:00–10:30.

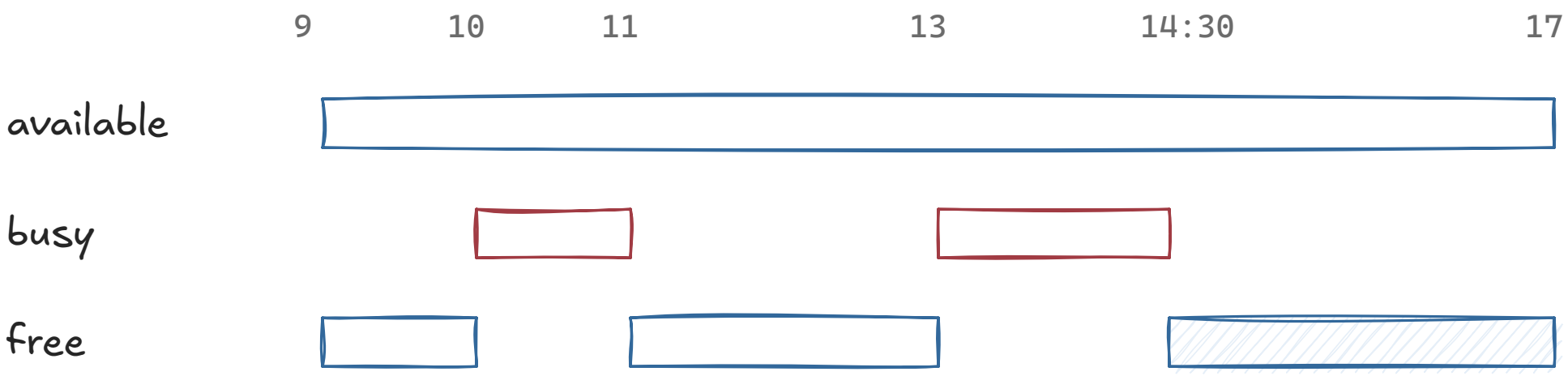


I'd use greedy when "good now" is useful. But I'd test rare-host cases like this. Fairer assignments later don't recover a meeting we already missed.

Fast intervals still need a fast data path.

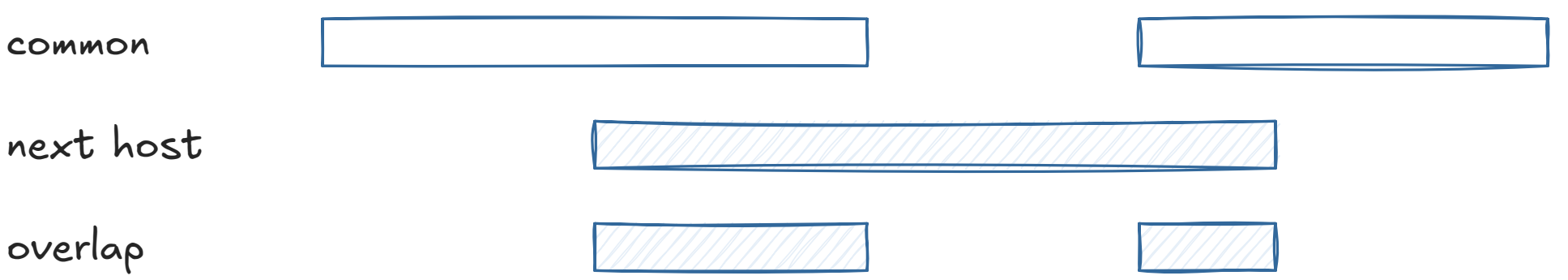
I like this part: make different rules look like the same kind of data.
Expand local schedules into UTC ranges once per day, not per slot.

Sort once, then keep the gaps.



A full daily limit becomes one busy range too. Same subtraction.
Sorting: O(n log n). A sorted sweep is linear; output adds work.

Everyone free? Walk two lists.



Keep the overlap. Advance whichever interval ends first.
O(m + n) for two sorted lists. No common time left? Stop.

Then check the whole request.

Batch database reads, group by host, and do the math in memory.
Never cache a failed calendar call as “free”.

```
warm: 30–75 ms     cold: ~600 ms     20-person page: ~3 s
```

Author’s Sep 2026 observations, not a speed guarantee.

I’d copy the interval representation and batched reads first. Then I’d measure the page people wait for, not just the math.

The post’s host-count cache is proposed work, not a finished result.

Source: Keith Williams / Cal.com / 9 Sep 2026